Replacing Keycloak's Infinispan Caches with Redis/Valkey

https://phasetwo.io
Phase Two

Garth Patil (gpatil@phasetwo.io)
Razvan Tufisi (rtufisi@phasetwo.io)
Keycloak Dev Day 2026

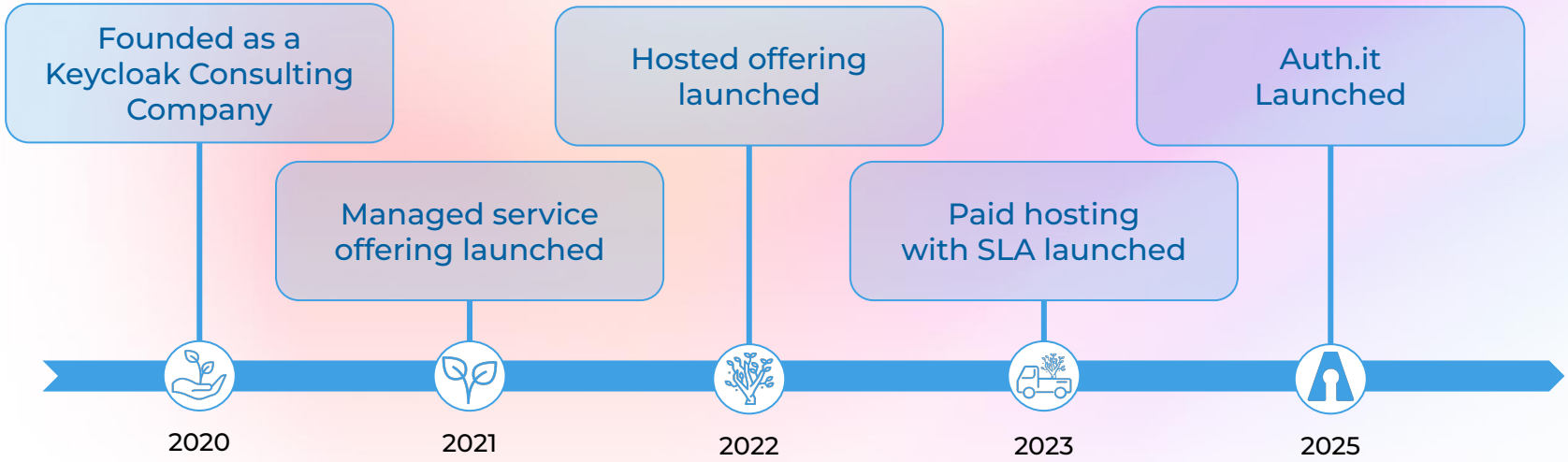
phase//

Auth.it

Who is Phase Two?

Keycloak extension, support and hosting.

Accelerate time-to-market and enterprise adoption for modern SaaS use cases.



phase//

Auth.it

Agenda

1. What our customers are saying about Infinispan
2. What are Redis and Valkey?
3. Goals and non-goals of our proof-of-concept
4. Implementation of replacing Infinispan distributed caches with Redis
5. Testing and benchmarking
6. Conclusions and next steps

What our customers think about Infinispan

- Infinispan has proven **operationally difficult** at all scales. This is/was especially true at large scale with either external or embedded Infinispan.
- **Failed restarts** or updates of Keycloak with embedded Infinispan because of JGroups incompatibility is a huge **red flag** and operational headache.
- **Split-brain** can happen spontaneously or because of misconfiguration.
- **Slow startup** because of JGroups/Infinispan discovery and rebalance **exacerbates downtime** associated with restarts and updates.

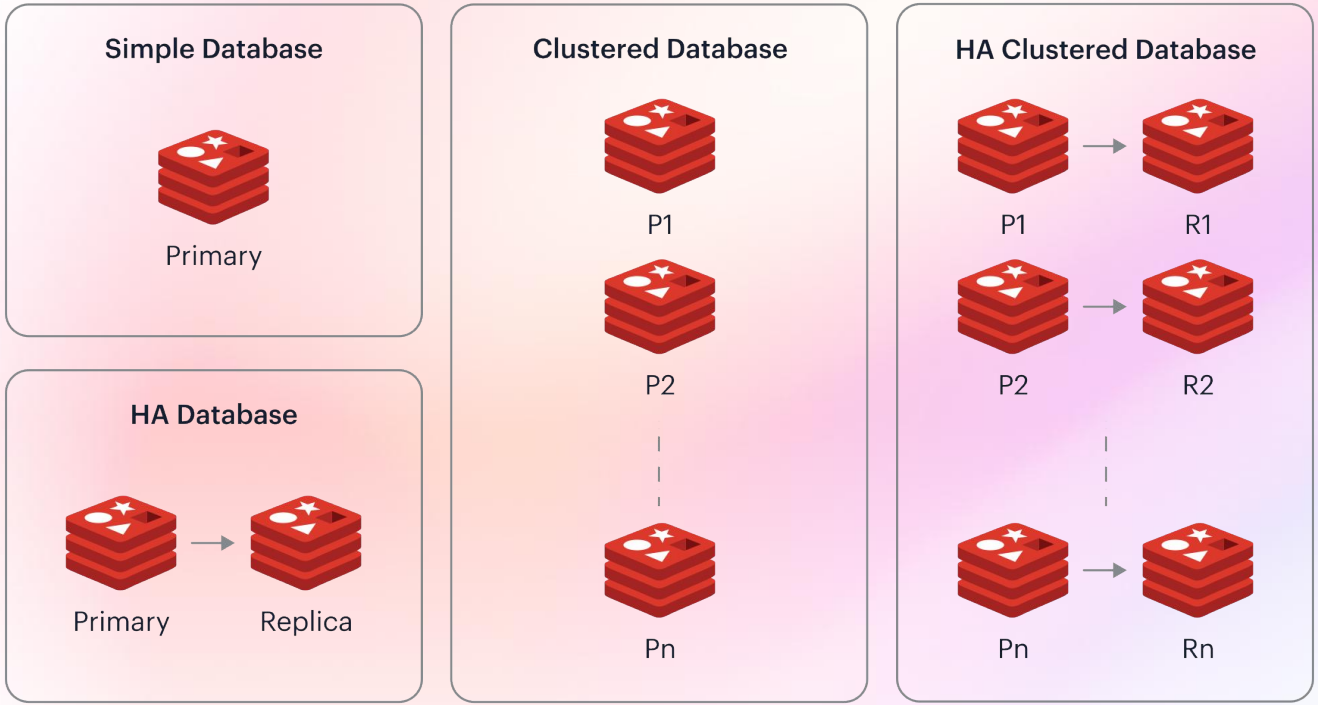
What our customers think about Infinispan

- Many customers need **fast, reliable, restarts**, as they are revising configuration, extensions and themes multiple times a day, and operating it via **CI/CD**.
- Keycloak operators frequently seek **simpler, managed** cache backends.
- It's 2026. Companies expect critical infrastructure to adhere to **cloud-native principles**, especially **no-downtime upgrades**.
- If they understood the deficiencies, complexities and operational costs that Infinispan brings with it, **they never would have chosen Keycloak**. 💩

What are Redis and Valkey?

- **In-memory** key–value data stores (store data in RAM for very fast access) **with persistence**
- Commonly used as **cache**, session store, message broker, and real-time data store
- Support advanced data types (strings, **hashes**, lists, **sets**, sorted sets, streams)
- Single-threaded core with **high performance and low latency**
- **Redis** – original, widely adopted open-source project
- **Valkey** – open-source fork of Redis, community-driven under the Linux Foundation

What are Redis and Valkey?



phase//

Auth.it

What are Redis and Valkey?

Standalone Topology

- Single node
- Simple to deploy and operate
- Replica for read-scaling and failover

Benefits

- Easy setup and management
- Lower infrastructure cost
- Predictable performance
- Minimal operational complexity
- Best for small deployments

Cluster Topology

- Multiple nodes sharing data via sharding
- Data automatically distributed across nodes (hash slots)
- Built-in replication and automatic failover
- Horizontally scalable

Benefits

- High availability (node failures tolerated)
- Scales out for larger datasets
- Handles higher throughput
- Better suited large-scale systems

How did we get here?

- The `DatastoreProvider` SPI makes it possible to replace the caches
- The **map-store** implementation is a great template for experiments
- The success of the **Cassandra** implementation is an inspiration
- We built a JPA implementation of the caches (prior to Persistent Sessions), so **we knew it was straightforward and possible**
- One of our customers asked, **"Why not Redis?"**

Goals

- **Functional parity** with existing implementations of distributed caches
- At least **50%** of the performance of Keycloak with embedded Infinispan
- **Fast startup** (<5s per pod) with **100% reliability**
- **Seamless upgrades**
- **Horizontal scaling** without rebalance issues
- Working with the current version of Keycloak, as an extension (**no forking**)

Non-goals

- Replacing **local** Infinispan caches (Realms, Users, etc.)
- Integration with or migration from **Persistent Sessions**
- Support all possible **Redis topologies**
- **Multi-region** (yet)

What we built

- Implementation of the `DatastoreProvider` that replaces the distributed caches.
- No-op implementations of several providers that cause JGroups and Infinispan to be used.
- A new **SPI for Redis connections** that is implemented using standalone or cluster mode connection clients.

What we built

- A simple `MapEntity` that could underlie all stored object types.
- A **changelog-based** `KeycloakTransaction` (similar to how Keycloak does it) for storing objects as hashes in Redis.
- A **Lua function** in Redis that simulates the same **check-and-set (CAS)** behavior of how entities are stored today.
- An implementation of `ClusterProvider` using **Redis** `PUBSUB` to propagate cache invalidation events and replace the Infinispan replicated `work` cache.

How we used Redis (data structures): Hash

What it is:

- A Redis **hash** stores a session as a key-value map (like a lightweight JSON object).
- Each entity type is stored under a single Redis key (e.g., `user-session:{sessionId}`).

How we use it:

- `HSET` → Store/update session fields (userId, clientId, expiration, metadata, etc.)
- `HGET` / `HGETALL` → Retrieve full session or specific attributes
- `HDEL`→ `Remove fields if needed`
- `EXPIRE` → Attach TTL for automatic session expiration

Why hashes:

- Efficient memory usage compared to full serialized blobs
- Partial updates without rewriting the whole session
- Fast constant-time field access (O(1))

How we used Redis (data structures): Set

Problem: Redis is key-based — you can't query by arbitrary fields like in a relational DB.

Solution: Secondary indexes using Sets

We maintain additional lookup keys:

- `SADD user:{userId}:sessions {sessionId}`
- `SADD client:{clientId}:sessions {sessionId}`

How we use them:

- `SADD` → Add session ID to index
- `SREM` → Remove session ID on logout/expiration
- `SMEMBERS` → Fetch all sessions for a user/client

Benefits:

- Fast lookup by user/client without scanning Redis
- Efficient bulk invalidation (e.g., logout all sessions for a user)
- Clean separation between storage (hash) and indexing (sets)

Implementation (MapEntity)

A Redis

Hash Storage

E.g.

```
{
  "offline": true,
  "userId":"8be4…",
  "brokerSessionId":
"a030…"
}
```

Sets
`user-session:realm-index:foo: ["1fea…", "28bc…"]`

B MapEntity

Just a `Map<String,String>` with dirty/deleted key tracking

E.g.

- `"offline": true`
- `"userId": "8be4…"`
- `"brokerSessionId": "a030…"`
- …

Secondary index requirement

```
abstract Map<String,
String>
getSecondaryIndexes();
```

C RedisUserSessionAdapter

Example concrete implementation with type conversion to/from strings

E.g.

```
@Override
public boolean isOffline() {
  return getBool("offline",
false);
}
```

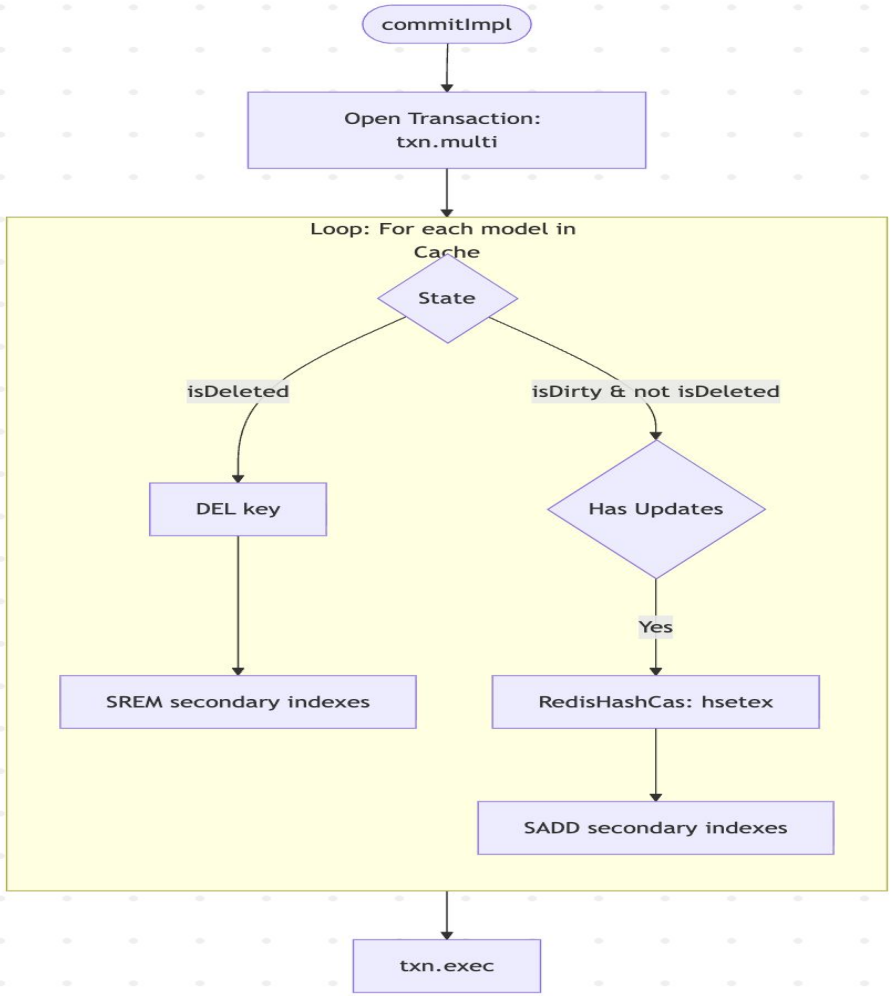
Implementation of secondary indexes that must be set on write.

phase//

Auth.it

Implementation (transaction)

- `RedisChangelogBasedTransaction` is how we batch updates in a `KeycloakTransaction`
- Eliminiates multiple reads for the same key per transaction
- Minimize writes and write sizes
  - Tracks changes and use partial `HSET`
- Bundle entity and secondary index writes
  - Use Redis `MULTI` and `EXEC` batches at the end reducing round trips



phase//

Auth.it

Implementation (atomic operations)

- `WATCH` = optimistic, multi-round-trip, retry-based, connection-scoped
  - High contention → frequent `EXEC` aborts
  - With `WATCH`, atomicity depends on a round-trip retry cycle.
  - Retry orchestration required in application code, amplifying load
- `EVAL` Lua check-and-set = atomic, single-round-trip, contention-resistant
  - Reduces latency, retry storms, and client complexity
  - Returns success/failure atomically
  - Lua eliminates race conditions and simplifies correctness guarantees
  - Better performance all around than `WATCH`

Implementation (expiration)

- Initial expiration was handled only by : `PEXPIREAT` in the LUA code we are executing in the `HSETEX` method
  - Benchmark tests revealed a issue `“Invalid refresh token”.` Some of the refresh token were looking for sessions that existed but no longer.
- Double-check in codebase to see if a cached entity expired and remove it.
- TTL handling on the Redis side might be wrong? 🤔

Implementation (issues)

- Logic to replicate what is happening in the Keycloak Infinispan implementation
  - There is a lot that happens here, especially with offline sessions, that is not documented, and must be done independently of the public interfaces

    AuthenticationProcessor.FIRST_OFFLINE_ACCESS

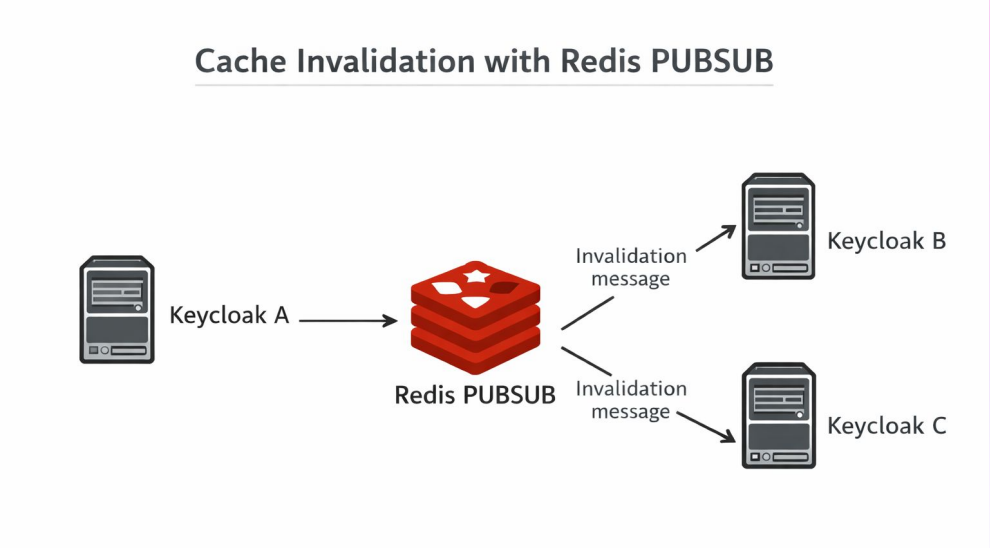
    https://github.com/keycloak/keycloak/discussions/36499#discussioncomment-12067317
  - Redis doesn't accept null values

    Null Sentinel value in Redis -> null Adapter
  - Mapping `AuthenticatedClientSessionKey`

    compound key: userSessionId + "::" + clientId

Implementation (messaging)

- `ClusterProvider` using `PUBSUB` that serializes all of the event types as JSON and propagates them to all nodes



Implementation (metrics)

- Metrics exposed in Keycloak's standard Prometheus endpoint with the `vendor_jedis_` prefix
  - Connection pool gauges (active|idle|wait), (created|borrowed|destroyed), (maxWait|meanIdle)
  - Redis command counters
  - Redis command/transaction timers

Testing

- Unit tests from **map-store** implementation (all pass)
- **Functional** test suite (currently sparse)
- **Benchmarks...**

Benchmarks

Benchmark (infrastructure setup)

Common configuration

- AWS EKS k8s cluster
- `m7g.xlarge` Graviton nodes
- 3 pods, 2GB, 2vCPU limits

Embedded Infinispan

- OOTB Keycloak, no changes

External Redis

- AWS ElastiCache - Redis OSS 7.1
- `cache.t4g.small` 1.3GB, 2vCPU
- 2 nodes: 1 primary, 1 replica

Benchmark (testing setup)

- Use **keycloak-bechmark**
  - `AuthorizationCode` test
  - **10,000 users**
  - **25 users per second**
  - **5 refreshes,**
  - **30s refresh period**
  - **60% logout rate**
  - **10 minute runtime**

```
./bin/kcb.sh \
 --scenario=keycloak.scenario.authentication.AuthorizationCode \
 --server-url=https://{keycloak-host} \
 --realm-name={realm} \
 --client-id={client} \
 --client-secret={secret} \
 --client-redirect-uri=https://example.com \
 --log-http-on-failure \
 --users-per-realm=10000 \
 --measurement=600 \
 --user-think-time=1 \
 --ramp-up=20 \
 --users-per-sec=25 \
 --logout-percentage=60 \
 --refresh-token-period=30 \
 --refresh-token-count=5 \
 --refresh-close-http-connection=true
```

Benchmark (results)

Redis

Requests	Executions					Response Time (ms)							
	Total	OK	KO	% KO	Cnt/s	Min	50th pct	75th pct	95th pct	99th pct	Max	Mean	Std Dev
All Requests	130999	130999	0	0.00	169.91	2	6	9	36	47	846	12	37
Browser to Log In Endpoint	15250	15250	0	0.00	19.78	15	20	23	33	86	846	27	102
Browser posts correct credenti...	15250	15250	0	0.00	19.78	25	34	38	48	57	111	35	6
Exchange Code	15250	15250	0	0.00	19.78	4	5	6	8	10	40	6	1
RefreshToken	76250	76250	0	0.00	98.9	4	6	6	8	12	98	6	2
Browser logout	8999	8999	0	0.00	11.67	2	3	3	5	9	79	3	2

Infinispan

Requests	Executions					Response Time (ms)							
	Total	OK	KO	% KO	Cnt/s	Min	50th pct	75th pct	95th pct	99th pct	Max	Mean	Std Dev
All Requests	131121	131121	0	0.00	170.07	3	15	20	51	67	336	19	15
Browser to Log In Endpoint	15250	15250	0	0.00	19.78	16	20	23	40	71	312	23	12
Browser posts correct credenti...	15250	15250	0	0.00	19.78	35	48	57	71	83	336	51	11
Exchange Code	15250	15250	0	0.00	19.78	3	5	6	9	22	254	6	6
RefreshToken	76250	76250	0	0.00	98.9	3	14	16	31	36	222	15	7
Browser logout	9121	9121	0	0.00	11.83	10	14	16	38	43	62	16	7

Benchmark (results)

	Infinispan 99th pct	Redis 99th pct	
All Requests	67	47	29.9%
Browser to Log In Endpoint	71	86	21.1%
Browser posts correct credenti…	83	57	31.3%
Exchange Code	22	10	54.5%
RefreshToken	36	12	66.7%
Browser logout	43	9	79.1%

Conclusions

- **Hit all goals 🎉🥳!! To review:**
  - ✅ Functional parity
  - ✅ At least 50% of the performance - **Hit 100%** (or better) for our benchmarks
  - ✅ Fast startup (<5s per pod) - **3s optimized image startup**
  - ✅ Seamless upgrades - Only tested with one version upgrade
  - ✅ No forking Keycloak

Conclusions

stianst on Feb 26, 2025 Maintainer

Keycloak uses caches for two purposes; local/embedded caches to prevent going to the database for everything, and secondly for non-persisted sessions that are stored in a distributed, but still embedded cache. Unless you are deploying a HA architecture of Keycloak the cache aka Infinispan is just an embedded part of Keycloak, and not something you have to worry about separately. In that regard it's simpler to stick with Infinispan than consider some external cloud managed cache service.

We have considered adding support for external caches so a Keycloak cluster can shared cached entities to prevent even more lookup in the database, but it's not something we prioritise at the moment since we believe the embedded caches work just fine.

Now, if you wanted to completely rip out Infinispan from Keycloak that would require a cache solution that is embeddable in a Java application; otherwise you just wouldn't get the same performance as we offer today. Honestly at this point you may just want to disable caching and rely more on the DB (which can also cache things on its own). It would be a huge effort, with little value add really, so not something we would consider (at least not without significant proof of it being both better and something that a wider part of the community would want). So, stick with Infinispan is my suggestion ;)

These assertions are **all wrong**.

phase//

Auth.it

Next steps

- Additional **performance optimization** for many usage patterns
- **Source** and **images** available (license TBD)
- **Multi-region** validation and documentation (via CockroachDB)
- **Phase Two Shared Hosting** clusters running it
- **“Serverless”** Mini-Keycloak

Special thanks...

- The Keycloak **maintainers**, **authors** and **contributors** for map-store and `DatastoreProvider`
- @dasniko & @srose for this **great event**
- @dominikschlosser for **keycloak-cassandra-extension**
- Our **excellent customers** who pushed us in this direction (Thank you **Gusto**!)
- Phase Two's **community contributors**
- **The entire Keycloak community**!

phase//

Auth.it

Questions?

THANK YOU!

More resources:

- Redis extension (links live soon!)
  - Image: https://quay.io/repository/phasetwo/keycloak-redis
  - Issues: https://github.com/p2-inc/keycloak-redis
- Phase Two: https://phasetwo.io
- Auth.it: https://auth.it
- GitHub: https://github.com/p2-inc

phase//

Auth.it